

Hill Point

Component Design Document

1 Description

Hill point attitude guidance. Computes the Hill frame attitude reference (attitude, rate, and acceleration of the orbit frame relative to inertial) from the spacecraft and primary body inertial states. Wraps the HillPointAlgorithm C++ algorithm via its C shim.

2 Requirements

No requirements have been specified for this component.

3 Design

3.1 At a Glance

Below is a list of useful parameters and statistics that give a quick look into the makeup of the component.

- **Execution** - *passive*
- **Number of Connectors** - 2
- **Number of Invokee Connectors** - 1
- **Number of Invoker Connectors** - 1
- **Number of Generic Connectors** - *None*
- **Number of Generic Types** - *None*
- **Number of Unconstrained Arrayed Connectors** - *None*
- **Number of Commands** - *None*
- **Number of Parameters** - *None*
- **Number of Events** - *None*
- **Number of Faults** - *None*
- **Number of Data Products** - *None*
- **Number of Data Dependencies** - 2
- **Number of Packets** - *None*

3.2 Diagram

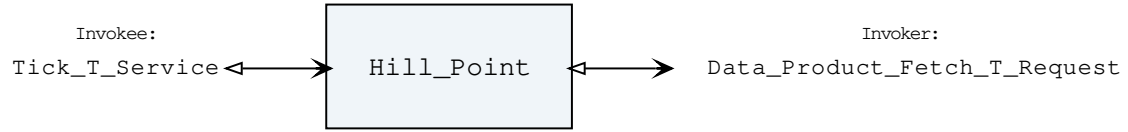


Figure 1: Hill Point component diagram.

3.3 Connectors

Below are tables listing the component's connectors.

3.3.1 Invokee Connectors

The following is a list of the component's *invokee* connectors:

Table 1: Hill Point Invokee Connectors

Name	Kind	Type	Return_Type	Count
Tick_T_Service	service	Tick.T	Att_Ref.T	1

Connector Descriptions:

- **Tick_T_Service** - Run the algorithm up to the current time and return the attitude reference it produces.

3.3.2 Invoker Connectors

The following is a list of the component's *invoker* connectors:

Table 2: Hill Point Invoker Connectors

Name	Kind	Type	Return_Type	Count
Data_Product_Fetch_T_Request	request	Data_Product_Fetch.T	Data_Product_Return.T	1

Connector Descriptions:

- **Data_Product_Fetch_T_Request** - Fetch a data product item from the database.

3.4 Interrupts

This component contains no interrupts.

3.5 Initialization

Below are details on how the component should be initialized in an assembly.

3.5.1 Component Instantiation

This component contains no instantiation parameters in its discriminant.

3.5.2 Component Base Initialization

This component contains no base class initialization, meaning there is no `init_Base` subprogram for this component.

3.5.3 Component Set ID Bases

This component contains no commands, events, packets, faults or data products that need base identifiers.

3.5.4 Component Map Data Dependencies

This component contains data dependencies that need their IDs mapped to valid data product IDs at initialization. Each data dependency also needs a stale limit provided in microseconds. If the timestamp of the data dependency is older than the reference time (usually the current time) minus the stale limit then the data dependency is considered too old to safely use. The `map_Data_Dependencies` procedure must be called with the following parameters:

Table 3: Hill Point Map Data Dependencies Parameters

Name	Type
Spacecraft_State_Id	Data_Product_Types.Data_Product_Id
Spacecraft_State_Stale_Limit	Ada.Real_Time.Time_Span
Primary_Body_State_Id	Data_Product_Types.Data_Product_Id
Primary_Body_State_Stale_Limit	Ada.Real_Time.Time_Span

Parameter Descriptions:

- **Spacecraft_State_Id** - The external data product ID used to fetch the value for the 'Spacecraft_State' data dependency.
- **Spacecraft_State_Stale_Limit** - The stale limit used for the 'Spacecraft_State' data dependency. If the fetched data product is older than the given Time_Span then it is deemed stale and unfit for use by the component.
- **Primary_Body_State_Id** - The external data product ID used to fetch the value for the 'Primary_Body_State' data dependency.
- **Primary_Body_State_Stale_Limit** - The stale limit used for the 'Primary_Body_State' data dependency. If the fetched data product is older than the given Time_Span then it is deemed stale and unfit for use by the component.

3.5.5 Component Implementation Initialization

The calling of this implementation class initialization procedure is mandatory. Initializes the Hill point algorithm. The `init` subprogram takes no parameters.

3.6 Commands

The Hill Point component has no commands.

3.7 Parameters

The Hill Point component has no parameters.

3.8 Events

The Hill Point component has no events.

3.9 Data Products

The Hill Point component has no data products.

3.10 Data Dependencies

Data dependencies for the Hill Point component.

Table 4: Hill Point Data Dependencies

Data Dependency Name	Type
Spacecraft_State	Cartesian_State.T
Primary_Body_State	Cartesian_State.T

Data Dependency Descriptions:

- **Spacecraft_State** - [m, m/s] Spacecraft inertial position and velocity in the inertial frame N.
- **Primary_Body_State** - [m, m/s] Inertial position and velocity of the primary body the orbit frame is defined about. Zero when the spacecraft state is already expressed relative to that body.

3.11 Packets

The Hill Point component has no packets.

3.12 Faults

The Hill Point component has no faults.

4 Unit Tests

The following section describes the unit test suites written to test the component.

4.1 *Hill_Point_Tests* Test Suite

This is a unit test suite for the Hill Point component

Test Descriptions:

- **Test** - Run the algorithm to ensure the Ada to C to C++ integration is sound.
- **Test_Invalid_Data_Dependency** - Ensure a data dependency with the wrong identifier is treated as a wiring defect and fails the tick's assertion.

5 Appendix

5.1 Preamble

This component contains no preamble code.

5.2 Packed Types

The following section outlines any complex data types used in the component in alphabetical order. This includes packed records and packed arrays that might be used as connector types, command arguments, event parameters, etc..

Att_Ref.T:

Table 5: Att_Ref Packed Record : 288 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Sigma_Rn	Packed_F32x3.T	-	96	0	95
Omega_Rn_N	Packed_F32x3.T	-	96	96	191
Domega_Rn_N	Packed_F32x3.T	-	96	192	287

Field Descriptions:

- **Sigma_Rn** - *No description provided.*
- **Omega_Rn_N** - *No description provided.*
- **Domega_Rn_N** - *No description provided.*

Cartesian_State.T:

Cartesian state containing position and velocity vectors.

Table 6: Cartesian_State Packed Record : 384 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Position	Packed_F64x3.T	-	192	0	191
Velocity	Packed_F64x3.T	-	192	192	383

Field Descriptions:

- **Position** - [m] Position vector
- **Velocity** - [m/s] Velocity vector

Data_Product.T:

Generic data product packet for holding arbitrary data types

Table 7: Data_Product Packed Record : 1096 bits (*maximum*)

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Data_Product_Header.T	-	72	0	71	-
Buffer	Data_Product_Types.Data_Product_Buffer_Type	-	1024	72	1095	Header.Buffer_Length

Field Descriptions:

- **Header** - The data product header

- **Buffer** - A buffer that contains the data product type

Data_Product_Fetch.T:

A packed record which holds information for a data product request.

Table 8: Data_Product_Fetch Packed Record : 16 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Data_Product_Types. Data_Product_Id	0 to 65535	16	0	15

Field Descriptions:

- **Id** - The data product identifier

Data_Product_Header.T:

Generic data_product packet for holding arbitrary data_product types

Table 9: Data_Product_Header Packed Record : 72 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Id	Data_Product_Types. Data_Product_Id	0 to 65535	16	48	63
Buffer_Length	Data_Product_ Types.Data_Product_ Buffer_Length_Type	0 to 128	8	64	71

Field Descriptions:

- **Time** - The timestamp for the data product item.
- **Id** - The data product identifier
- **Buffer_Length** - The number of bytes used in the data product buffer

Data_Product_Return.T:

This record holds data returned from a data product fetch request.

Table 10: Data_Product_Return Packed Record : 1104 bits (*maximum*)

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
The_ Status	Data_ Product_ Enums. Fetch_ Status.E	0 => Success 1 => Not_Available 2 => Id_Out_Of_Range	8	0	7	-
The_ Data_ Product	Data_ Product. T	-	1096	8	1103	-

Field Descriptions:

- **The_Status** - A status relating whether or not the data product fetch was successful or not.
- **The_Data_Product** - The data product item returned.

Packed_F32x3.T:

Packed 3 element vector of 32-bit floats.

Table 11: Packed_F32x3 Packed Array : 96 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Short_Float	-3.40282e+38 to 3.40282e+38	32	3	96

Packed_F64x3.T:

Packed 3 element vector of 64-bit floats.

Table 12: Packed_F64x3 Packed Array : 192 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Long_Float	-inf to inf	64	3	192

Sys_Time.T:

A record which holds a time stamp using EMA VTC format including seconds and subseconds since epoch (1-5-1980 to 1-6-1980 midnight).

Table 13: Sys_Time Packed Record : 48 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Seconds	Interfaces. Unsigned_32	0 to 4294967295	32	0	31
Subseconds	Interfaces. Unsigned_16	0 to 65535	16	32	47

Field Descriptions:

- **Seconds** - The number of seconds elapsed since epoch.
- **Subseconds** - The number of $1/(2^{16})$ sub-seconds.

Tick.T:

The tick datatype used for periodic scheduling. Included in this type is the Time associated with a tick and a count.

Table 14: Tick Packed Record : 80 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Count	Interfaces. Unsigned_32	0 to 4294967295	32	48	79

Field Descriptions:

- **Time** - The timestamp associated with the tick.
- **Count** - The cycle number of the tick.

5.3 Enumerations

The following section outlines any enumerations used in the component.

Data_Product_Enums.Fetch_Status.E:

This status denotes whether a data product fetch was successful.

Table 15: Fetch_Status Literals:

Name	Value	Description
Success	0	The data product was returned successfully.
Not_Available	1	No data product is yet available for the provided id.
Id_Out_Of_Range	2	The data product id was out of range.